

Report of Fakeddit Data Analysis

Introduction and motivation

The problem of fake news dissemination is acute in modern society, directly threatening citizens' trust in institutions and provoking irrational panic during critical moments such as medical epidemics. Combating this threat requires reliable machine learning algorithms, yet their development is seriously hindered by a shortage of high-quality data. Most traditional datasets, such as LIAR or Some-Like-It-Hoax, have very substantial limitations that make them largely unsuitable for comprehensive analysis. As a rule, they are too small in volume, restricted exclusively to text format and binary classification logic (where a news item can only be true or false), and focused on very narrow subject areas, such as American politics, which prevents models from learning from diverse everyday content.

Dataset Characteristics

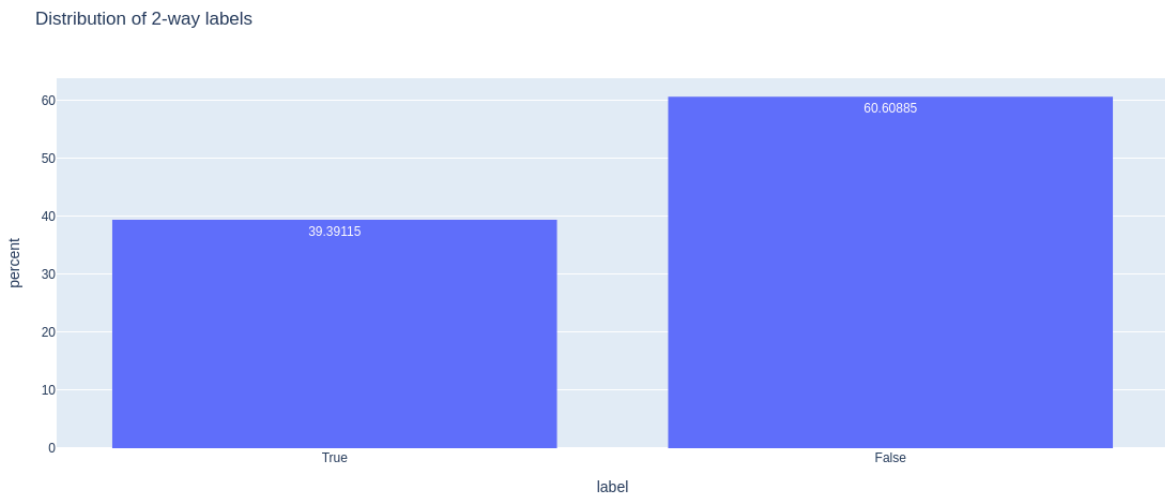
The uniqueness of the Fakeddit dataset lies in its multimodality, which allows algorithms to rely not only on isolated text but also on the visual context of images, significantly enriching the amount of information that can be used for the detection of misinformation. The data consists of post titles, images associated with them and additional metadata such as creation time, number of comments, subreddit name, etc. The authors of the original dataset carefully extracted not only the headlines and attached images but also rich metadata, including scores, author names, and audience comments.

The dataset consists of over 1 million posts, of which around 682k have an image associated with them. The posts come from across cleanest and most relevant 22 subreddits. The overall quality of the final general sample was good enough, no further pre-processing was required over the one provided by the dataset authors. During data collection there were also additional quality assurance steps, which can be reduced to all posts with score < 1 , and subreddit-specific title tags removing in practice.

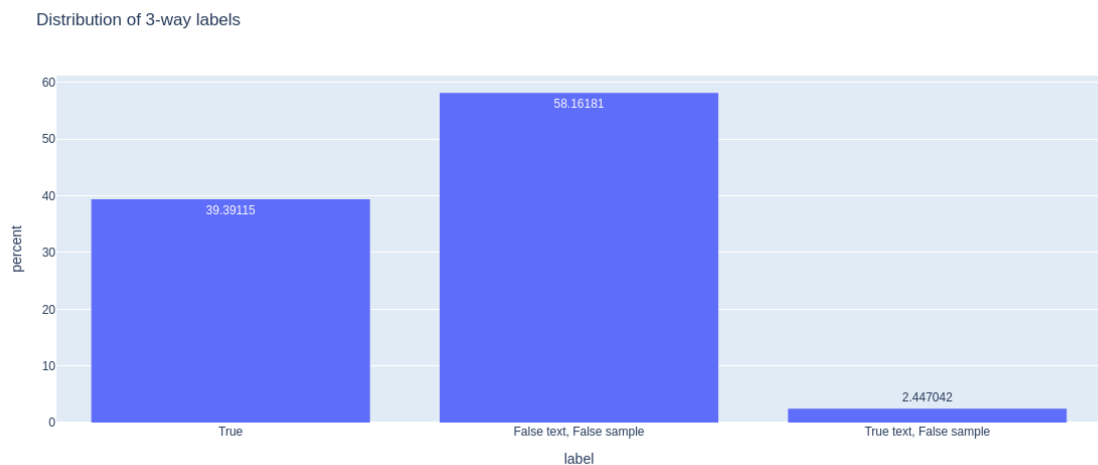
Labels of The Posts

Labeling the vast amount of data was performed using the distant supervision method (an approach that involves automatically assigning labels to all posts based on the known general theme of the subreddit from which they were extracted). This eliminates the need to manually label each individual post. Thus, for each sample there are three different labels assigned, corresponding to different levels of granularity. We will describe it here vividly, but concisely:

- 2-way labels: a **True/False** split - represents whether or not the posts presents factual information

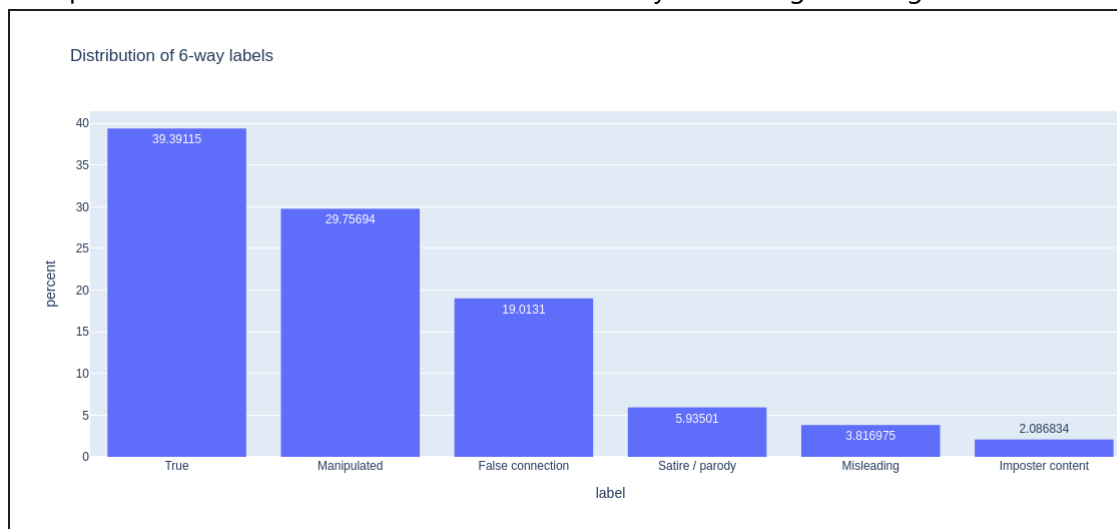


- 3-way labels: posts containing false data have been split into two categories:
 - false text, false sample - sample has **False** label in 2-way labeling and contains untrue information
 - true text, false sample - sample has been deemed **False**, but contains "true" text - it involves only posts containing propaganda posters.



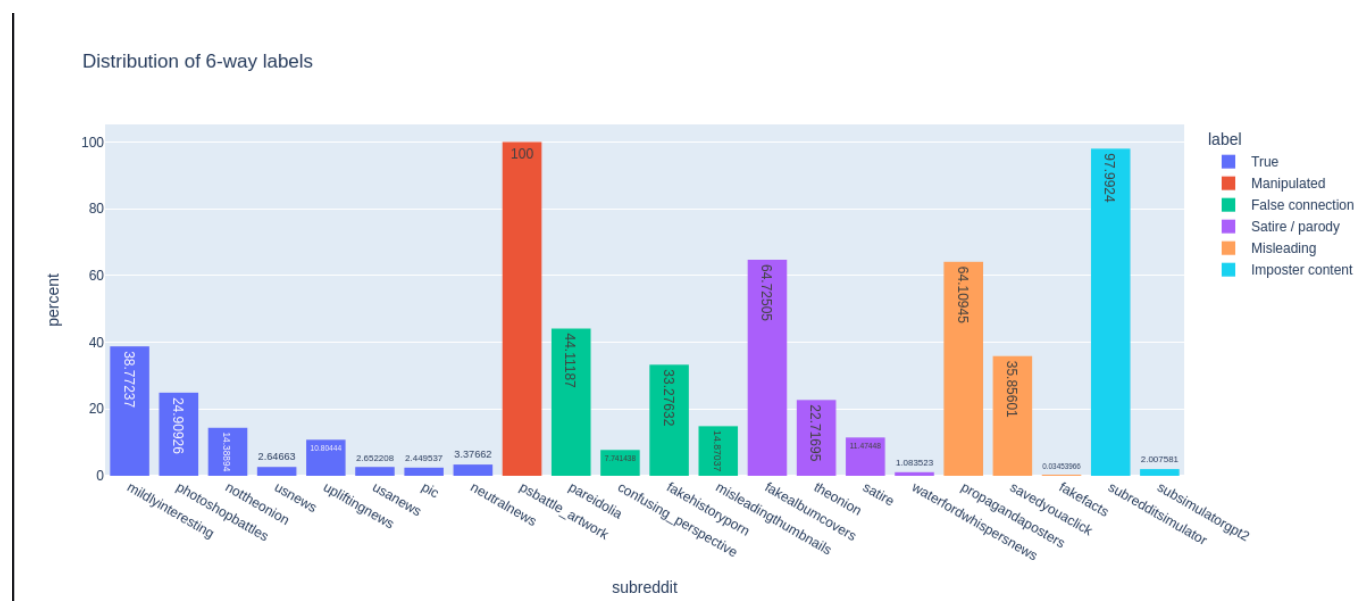
- 6-way labels: **True** label remains the same as in the other two kinds of labeling, the **False** label has been split into:
 - imposter content; i.e. content auto-generated by bots
 - misleading content: content conveyed in a manipulated way to mislead people in believing false narratives
 - satire / parody
 - false connection: submission images in this category do not accurately support their text descriptions

- manipulated content: content that has been manually altered e.g. via image modifications



It should be noted that some of these categories, in particular Manipulated Content, consist almost entirely of visually altered pictures, so when creating narrowly focused models that analyse only textual data, it is advisable to disregard such labels so as not to introduce unnecessary mathematical noise into the training sample. The experiments from the original authors showed that combining text and images works much better than using just one, with the winning setup being BERT plus ResNet50 merged via max pooling. Error analysis revealed that models struggle the most with bot-generated content and satire (which requires real-world knowledge), while manipulated images are surprisingly easy to catch though a bias toward the "True" label remains due to class imbalance.

For each subreddit the authors of chosen dataset they've inspected 10 random samples. If the 6-way label they assigned for each of the sample matched, so then the subreddit was kept in the dataset and the label was assigned to each post in the subreddit. This means the misinformation classification task turns into just predicting the subreddit based on the title alone.



A curious example is the label "manipulated" for the subreddit psbattle_artwork. That subreddit does contain image manipulation, but the posts there are mostly satirical in nature.

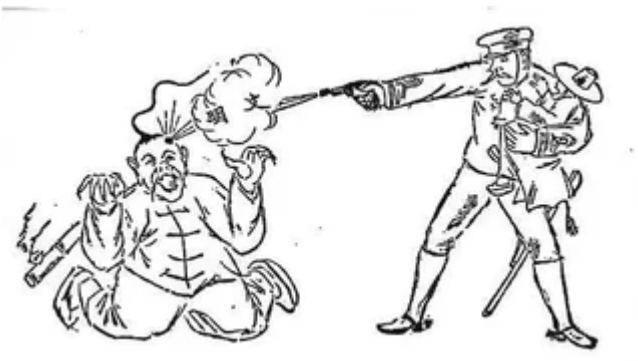
Some Indicative Examples



Above is a picture of a cat titled "he looks huge". This post comes from [confusing_perspective](#) subreddit - thus it's labeled as false connection.



The above post is titled "Obligatory godzilla post" and comes from [psbattle_artwork](#) subreddit.



Cleaned title: "newspaper illustration depicting japan shooting china with the bullet of civilization first sinojapanese war"



Another propaganda poster "roosevelt figured wrong the tentacles of the dollaroctopus will be cut the jews in the white house and the gold in fort knox have been surrounded by young people by the armies of labor nsb poster from the netherlands"

Proposed Pipeline & Workflow

To formalise our project, we need to describe the structure and order of data manipulations. The creators of the original Fakeddit were extremely concise in their assessments. Their publication contains almost no detailed steps or conventions for handling intermediate results. The final metrics are presented rather sparingly, which complicates direct analysis. Therefore, this project offers an alternative approach. It is not merely a technical replication but an attempt to deepen and conceptually rethink the conclusions of the benchmark authors. The main business task is to independently verify the results obtained by original authors. We strive to build alternative methods for constructing classifiers and to see whether these new, more modern approaches can surpass the original or provide serious competition in real world conditions. The implementation of our plan differs qualitatively from that proposed by the original due to strict objective constraints. The main challenge was the colossal size of the image database, exceeding 100 gigabytes. Operating with such multimedia arrays on basic computing resources is technically extremely difficult. However, despite of it, we deliberately decided to raise the complexity bar even more, abandoning the initially considered simplified version of the dataset containing only 55k items. We took instead a huge sample from the original dataset crafted on Kaggle, comprising about 700k records. An important consequence of this decision became a key feature of our pipeline. **The vast majority of analytical work is carried out exclusively on textual data.** Adding visual content became just and exclusively the final stage of our project.

Given the stated business goals and constraints, we built a strict sequence of dataset analysis consisting of five evolving stages. Each subsequent step logically follows from the limitations and problems of the previous one. The first stage involves exploratory data analysis (EDA) and data cleaning. Without this step, any metrics would

be distorted by information noise, so obtaining a clean base of 700k items became the foundation for all further actions. In the second stage we create baseline models based on TF-IDF. The business always needs a starting point, the simplest and fastest algorithm against which all expensive and complex solutions will be compared. However, the baseline relies on primitive word counting and completely ignores the semantics of language. Understanding this conceptual limitation naturally leads to the third stage: testing advanced ensemble models *in combination with modern language transformers*. Here we use embedders to extract deep meaning from texts and feed these complex data to algorithms such as LightGBM. When we find the most successful combination of model and embedder, the issue of tuning is being considered, because default algorithm settings do not always allow extracting maximum efficiency. So, in the next step the Optuna framework basically tune hyperparameters to the specific geometry of our analysed information. Finally, as we are aware, that text alone models cannot evaluate properly the situation with false connection or tricky text-picture interplay, the Vision-Language Model was considered and used on the data.

Each stage of our pipeline is driven by a set of general assumptions (*or hypotheses*). First, we expect to **achieve results close to those obtained in the original study**, assuming that our overwhelming focus on textual data in the first four stages will not lead to a radical drop in metrics compared to the overall benchmark. Second, we test the hypothesis that **alternative modern text transformers will be able to demonstrate significantly better results**, even taking into account our computational constraints. We hope that the theoretical effectiveness of these complex tools will be unequivocally proven in practice, because otherwise the feasibility of their implementation in real business processes would be highly questioned. Finally, the original paper leaves some ambiguity regarding how critical the presence of images is for the claimed fine-grained classification. Recognising the specificity of the Fakeddit dataset, we assume the some particular importance of visual content. Therefore, our final business hypothesis is that **the image-based classification model will show a result at least comparable to the best purely text-based model**, and ideally surpass it.

Theoretical Fundamentals

Within our project, the processing of Fakeddit dataset could be understood as supervised learning task. We aim to build a model that uncovers hidden patterns and predicts one of six categories for new posts. From this perspective, it should be reminded, that classification technique basically finds separating boundaries in a described feature space. Since we have six classes, we use multiclass strategies such as One-vs-Rest or native multiclass algorithms. Also before mere modelling, we perform Exploratory Data Analysis (EDA) using descriptive statistics and visualizations to detect class imbalance, text length distributions, and potential outliers. Strong imbalance would bias predictions toward the majority class. Data cleaning is critical to prevent data leakage problems, where test information accidentally enters training (e.g., duplicate posts). As a part of data cleaning, we remove rows with critical missing information rather than imputing them, considering the fact that synthetic values often distort semantic meaning to protect mathematical and semantic purity of the training sample.

To establish a meaningful benchmark, we begin with a simple and interpretable baseline: TF-IDF vectorization combined with logistic regression. TF-IDF converts text into a sparse numerical matrix by counting term frequencies and penalizing words that appear too often across the corpus, thereby highlighting truly distinctive terms. Logistic regression then takes these vectors, computes a weighted sum of features, and passes it through a sigmoid function to produce a probability of a post being fake. This baseline is mathematically transparent, fast, and allows us to directly inspect which words the model considers deceptive. But, linear models often fail to capture non-linear patterns in language, so we turn to ensemble methods,

specifically Random Forest and LightGBM. Random Forest builds many independent decision trees in parallel, reducing variance and overfitting. LightGBM, a gradient boosting algorithm, sequentially adds trees that correct the errors of previous ones by approximating the antigradient of the loss function. Its histogram-based discretization and leaf-wise tree growth (always splitting the leaf that gives the largest error reduction) make it exceptionally fast and accurate on large datasets.

It should be said that fundamental limitation of TF-IDF is that it ignores word order and context, treating each text as a mere bag of words. To capture semantic meaning, we use language embeddings from SentenceTransformers. These dense vectors encode entire sentences such that geometrically close vectors indicate semantically similar texts, even without shared vocabulary. We employ lightweight models like all-MiniLM-L12-v2 and paraphrase-MiniLM-L3-v2, which are products of knowledge distillation. They mimic large neural networks but have far fewer parameters, making them fast in some sense. The paraphrase family is particularly valuable for our project because it recognises hidden manipulation even when fake news authors rephrase known false narratives. After obtaining these semantic features, we need to ensure our classifiers perform at their best. Instead of relying on default hyperparameters, we use Optuna, a Bayesian optimization framework based on the TPE (Tree-structured Parzen Estimator) algorithm. Optuna does not blindly search but mathematically analyses previous trials to select the most promising hyperparameter values. For our baseline, we tune the TF-IDF parameters (max_df, min_df) and the logistic regression regularization strength. For LightGBM, we optimise the number of trees, learning rate, tree depth, and leaf count.

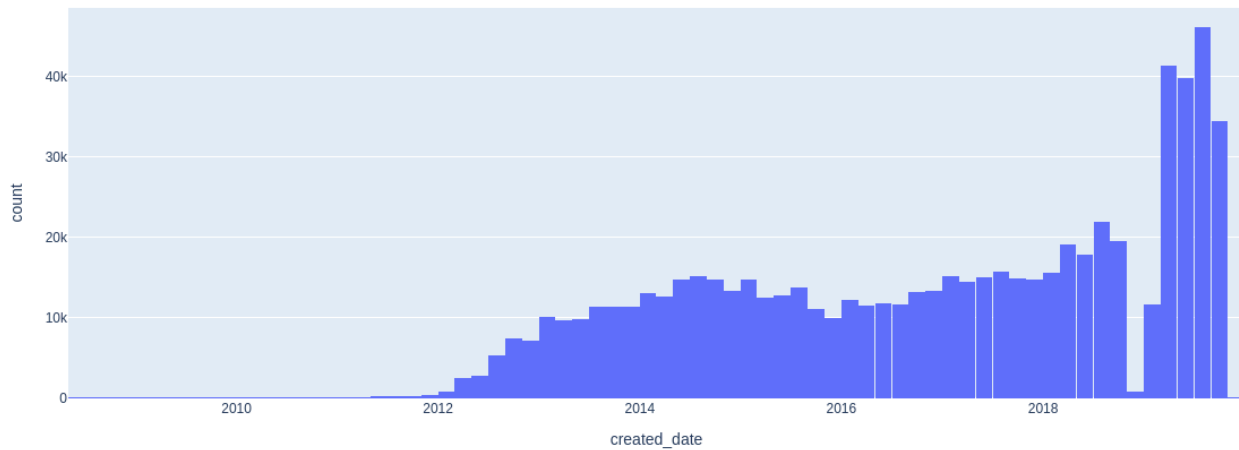
The final theoretical step in our project is the shift from text-only to multimodal analysis, mainly because of the percentage of unused visual contents before this stage. That's why we try here modern Vision-Language Models (VLMs), which go far beyond traditional convolutional networks by deeply integrating visual and textual perception. At their core are so-called Vision Transformers that split an image into small patches, convert them into numbers, and project them into the same hidden space as text tokens. Cross-attention layers allow the model to compare written words with specific objects in the image. In practice we feed the model both an image and the post's headline simultaneously, then extracting the last layer's hidden states, applying an attention mask to filter padding, and average the results to obtain a single fused vector that combines the meaning of text and image in the end. This fused representation can be theoretically crucial because of the detection enabling of the most difficult category, False Connection, where a truthful text and a ambiguous photo are deliberately paired to create a some type of compromised story.

Step I. Introductory EDA & Cleaning

Temporal resolution

The dataset contains data from quite large date range. The first posts come from 2008, while the last ones from 2019.

Post distribution over time



Splitting the data

For the purpose of the training we have split the data using 60/20/20 stratified split. The usage of temporal split also seemed justified initially, but the labeling strategy used by the dataset author makes it so that time is not main source of bias. For example: post in the **usnews** subreddit are automatically assigned as truthful, regardless of the date.

Provided metadata

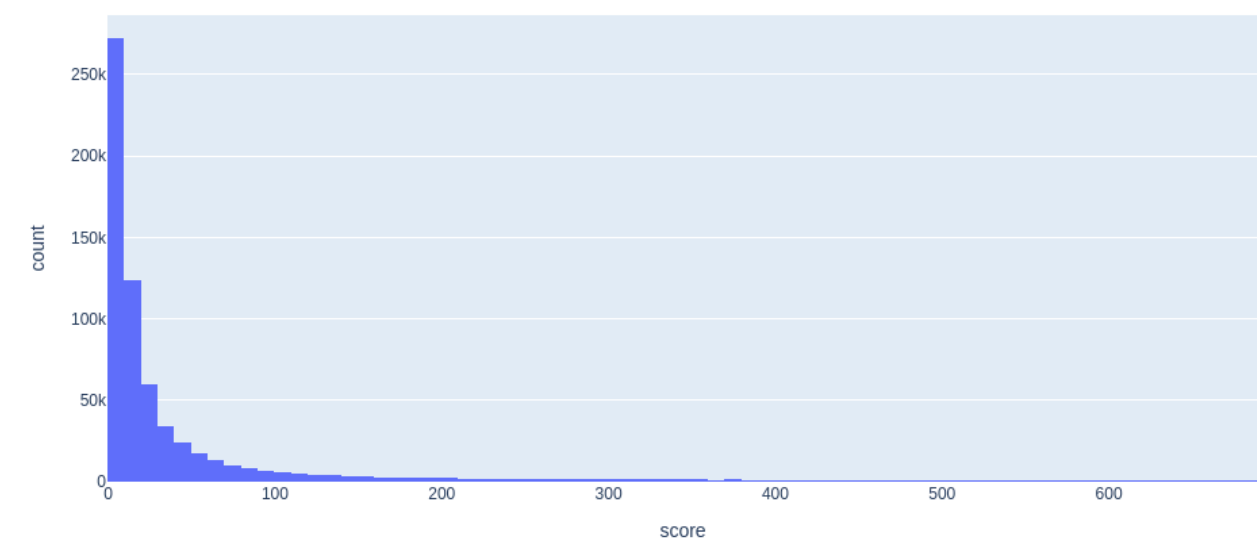
The dataset provides also information on:

- author (almost 35k different values!)
- scores (upvotes - downvotes)
- upvote ratio
- number of comments
- etc

column	null_count	null_pct
str	u32	f64
"author"	34841	0.051037
"clean_title"	0	0.0
"created_utc"	0	0.0
"domain"	203139	0.297569
"hasImage"	0	0.0
"id"	0	0.0
"image_url"	1863	0.002729
"linked_submission_id"	479522	0.702431
"num_comments"	203139	0.297569
"score"	0	0.0
"subreddit"	0	0.0
"title"	0	0.0
"upvote_ratio"	203139	0.297569
"2_way_label"	0	0.0
"3_way_label"	0	0.0
"6_way_label"	0	0.0
"created_date"	0	0.0

In there were some null values. There was no choice of clearning strategy since we did not use them for the training. Utilizing these could be a future direction.

Distribution of scores for subreddit posts



Using metadata is also tricky, for example authors mostly appear once or twice, although there are some clear

statistic	author	post_count
str	str	f64
"count"	"274605"	274606.0
"null_count"	"1"	0.0
"mean"	null	2.485965
"std"	null	76.171882
"min"	"-----_--"	1.0
"25%"	null	1.0
"50%"	null	1.0
"75%"	null	2.0
"max"	"zzzzz94"	34841.0

outlines - which could bring some signal.

Step II. The Power and Unility of Just-Text Models

For our initial experiments we have only used one column of the dataset `clean_title`. It consists of cleaned post titles as described previously.

Baseline model

We have started by a simple model using combination of `Tf-Idf` embdedings and a logistic regression model. It is based on uni and bigrams.

Binary case

First we've wanted to see performance on 2-way labels to gain better intuition on how the preditions are made:

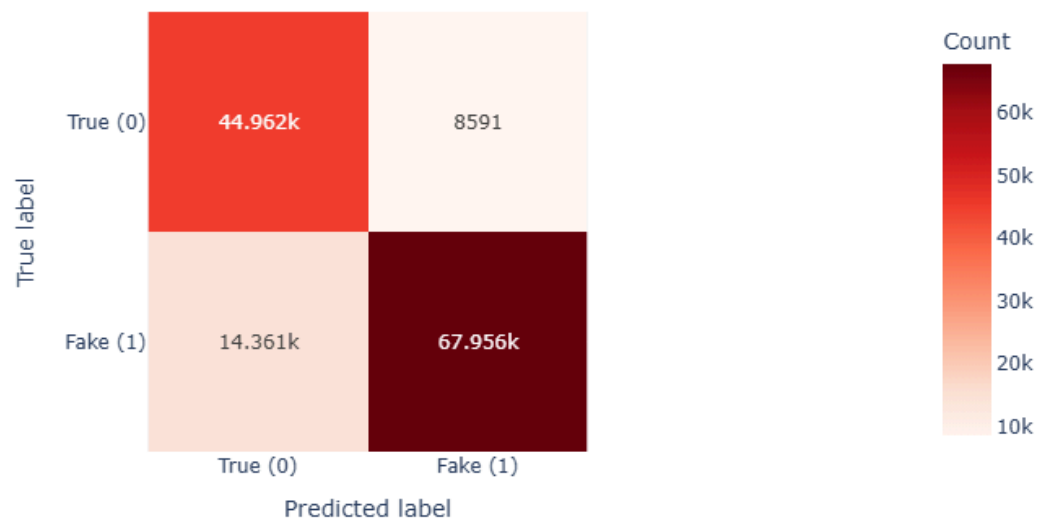
Test set results: ROC AUC (Binary): 0.9085

	precision	recall	f1-score	support
True (0)	0.7579	0.8396	0.7967	53553
Fake (1)	0.8878	0.8255	0.8555	82317
accuracy			0.8311	135870

rank	True	Fake
1	says	cutouts
2	in	circa
3	police	other discussions

rank	True	Fake
4	donates	discussions
5	sign	mrw
6	lets you	colourized
7	saves	til
8	way the	florida man
9	tells	colorised
10	these	poster

Confusion Matrix - Binary Model



We see that model is biased towards the most common label (False).

We get an interesting overview by the ranking of words that are the most impactful for the predictions.

To get more context we took a look at the most common words for each subreddit then:

```
[30...] shape: (22, 11)
```

subreddit	rank1	rank2	rank3	rank4	rank5	rank6	rank7	rank8	rank9	rank10
str	str	str	str	str	str	str	str	str	str	str
"confusing_perspective"	"the"	"this"	"like"	"looks"	"and"	"with"	"giant"	"look"	"dog"	"cat"
"fakealbumcovers"	"the"	"and"	"you"	"self"	"titled"	"this"	"for"	"album"	"selftitled"	"with"
"fakefacts"	"the"	"its"	"and"	"that"	"know"	"actually"	"size"	"fake"	"facts"	"had"
"fakehistoryporn"	"the"	"circa"	"his"	"and"	"for"	"after"	"first"	"from"	"before"	"with"
"mildlyinteresting"	"the"	"this"	"and"	"like"	"has"	"found"	"that"	"with"	"looks"	"for"
"misleadingthumbnails"	"the"	"this"	"with"	"new"	"from"	"and"	"for"	"out"	"man"	"dog"
"neutralnews"	"trump"	"the"	"for"	"and"	"says"	"with"	"after"	"from"	"new"	"over"
"nottheonion"	"for"	"the"	"man"	"with"	"after"	"and"	"police"	"from"	"says"	"woman"
"pareidolia"	"this"	"the"	"happy"	"like"	"looks"	"you"	"guy"	"little"	"was"	"and"
"photoshopbattles"	"this"	"the"	"with"	"cat"	"dog"	"and"	"his"	"man"	"guy"	"from"
"pic"	"the"	"and"	"this"	"from"	"sunset"	"park"	"lake"	"new"	"for"	"national"
"propagandaposters"	"the"	"poster"	"and"	"for"	"war"	"from"	"you"	"soviet"	"with"	"ussr"
"psbattle_artwork"	"the"	"you"	"this"	"other"	"discussions"	"cutouts"	"and"	"for"	"here"	"not"
"satire"	"the"	"for"	"trump"	"and"	"with"	"new"	"after"	"from"	"man"	"you"
"savedyouaclck"	"the"	"this"	"you"	"and"	"what"	"why"	"for"	"how"	"are"	"one"
"subredditsimulator"	"the"	"and"	"this"	"for"	"with"	"you"	"that"	"was"	"from"	"have"
"subsimulatorgpt2"	"the"	"and"	"this"	"with"	"for"	"you"	"was"	"that"	"not"	"skeltal"
"theonion"	"the"	"for"	"and"	"with"	"this"	"that"	"life"	"you"	"have"	"out"
"upliftingnews"	"the"	"for"	"and"	"with"	"from"	"after"	"his"	"man"	"who"	"her"
"usanews"	"the"	"for"	"and"	"trump"	"with"	"new"	"after"	"from"	"police"	"are"
"usnews"	"the"	"trump"	"for"	"and"	"with"	"after"	"from"	"new"	"says"	"man"
"waterfordwhispersnews"	"the"	"for"	"local"	"man"	"new"	"you"	"from"	"with"	"this"	"about"

Especially for the fake case we may see:

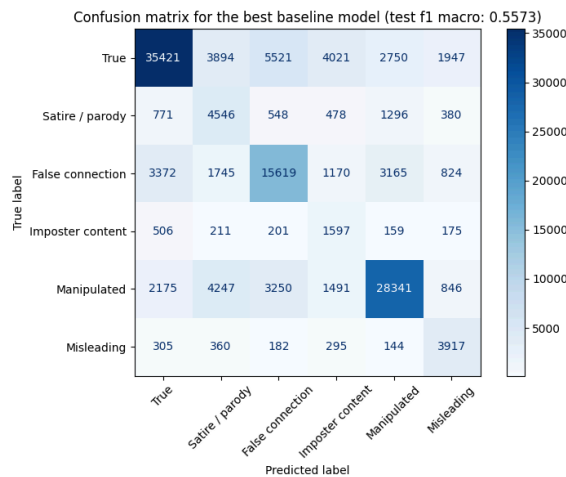
- **circa**, **colourized**, **colorised** - words that are most probably associated with **fakehistoryporn** subreddit
- **other**, **discussions** - words that are common in **psbattle_artwork** subreddit

Hence we may deduce that terms frequency provides information about the subreddit which contributes an indirect data leakage. Although the Tf

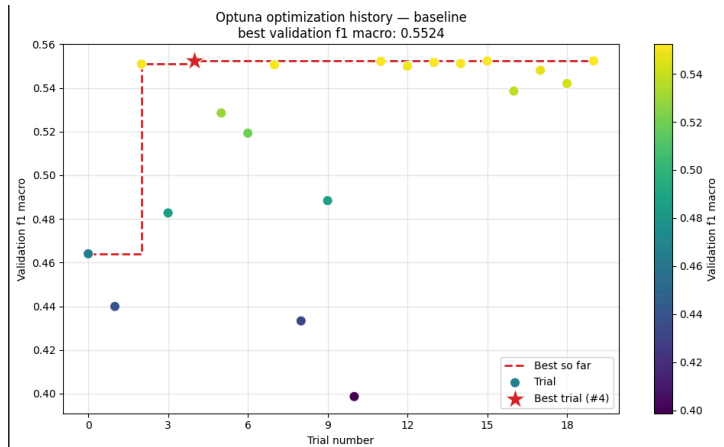
6-way case

Afterwards we trained the same combination of Tf-Idf embeddings and logistic regression for the 6-way case. In that case Optuna has also been used for hyperparameter optimization for 20 trials.

Confusion Matrix



Optuna training history



And the word rankings:

	rank	True	Satire / parody	False connection	Imposter content	Manipulated	Misleading
0	1	lets you	questions with	circa	mrw	cutouts	poster
1	2	uplifting	must see	colourized	til other	discussions	posters
2	3	says	blog	colorised	fwd	discussions	ussr
3	4	donates	patriothole	bc	florida man	obligatory	usa
4	5	jumping	quiz	bce	hmb	available here	leaflet
5	6	odometer	news	happy to	ysk	had to	date unknown
6	7	this	selftitled	decolorized	montage	photoshop	soviet
7	8	lawmaker	announced that	rfakehistoryporn	lpt	swap	wwii
8	9	way the	satire	pareidolia	discussion	michael bay	jap
9	10	shaped	ftw	little guy	florida woman	imgur	pamphlet

Here we have another finding; in the **misleading** category containing **propagandaposters** subreddit, "poster", "posters" and "ussr" are the words that influence the prediction the most.

Semantic Embeddings

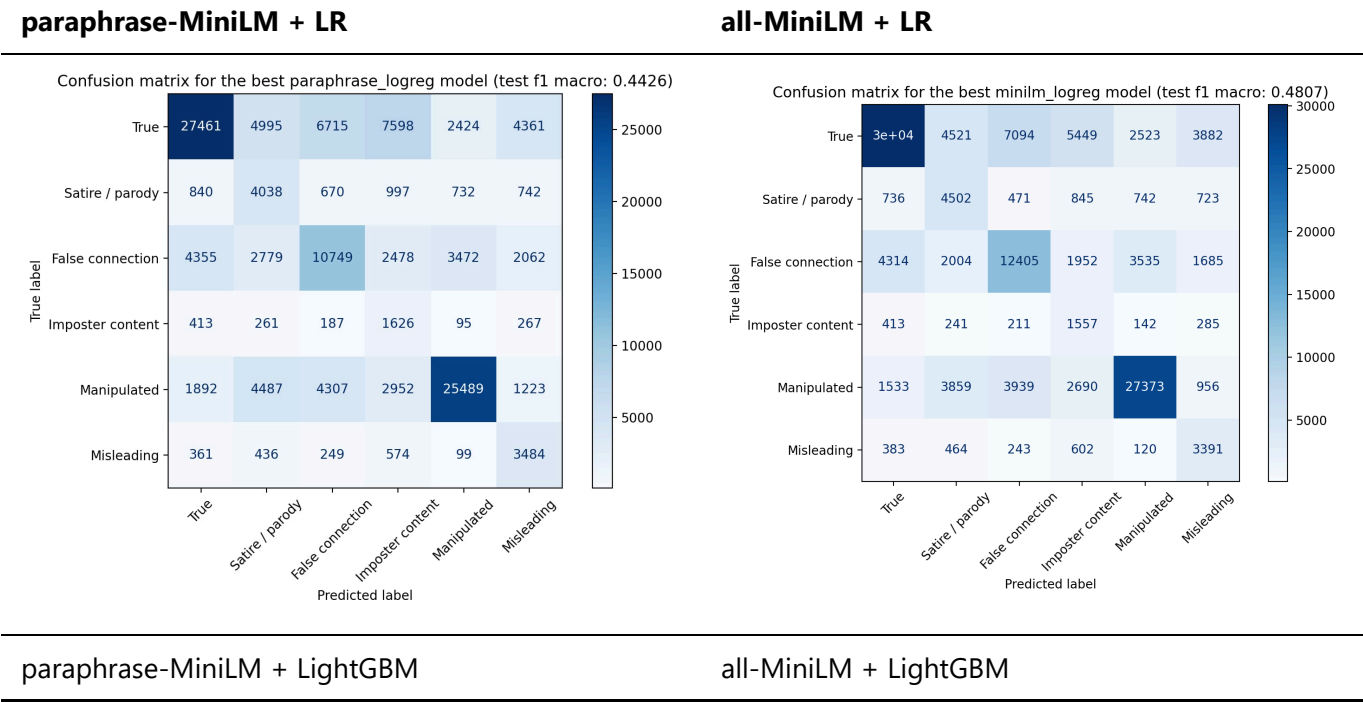
Afterwards we have tried to use more general embeddings from **sentence_transformers** library. We have used two text embedding models:

- **paraphrase-MiniLM-L3-v2**
- **all-MiniLM-L12-v2** both based on transformer architecture. Their choice was based on the benchmark on **sentence_transformers** documentation.

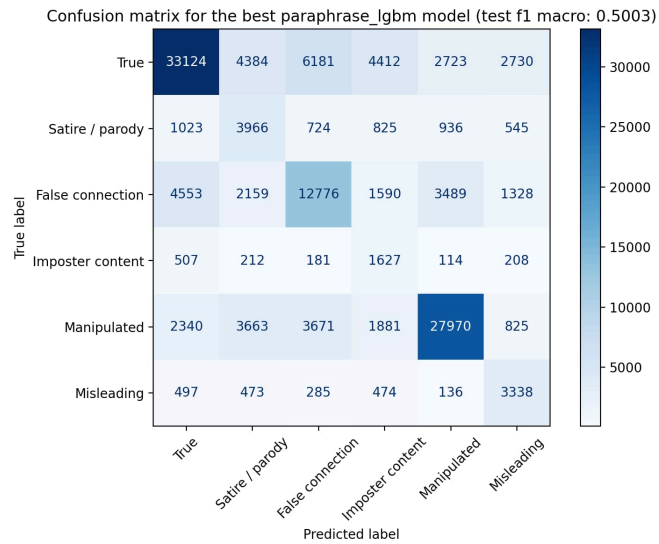
All models 					
Model Name	 Performance Sentence Embeddings (14 Datasets) 	Performance Semantic Search (6 Datasets) 	Avg. Performance 	Speed 	Model Size 
sentence-transformers/all-mpnet-base-v2 	69.57	57.02	63.30	2800	420 MB
sentence-transformers/all-distilroberta-v1 	68.73	50.94	59.84	4000	290 MB
sentence-transformers/all-MiniLM-L12-v2 	68.70	50.82	59.76	7500	120 MB
sentence-transformers/all-MiniLM-L6-v2 	68.06	49.54	58.80	14200	80 MB
sentence-transformers/multi-qa-mpnet-base-dot-v1 	66.76	57.60	62.18	2800	420 MB
sentence-transformers/multi-qa-distilbert-cos-v1 	65.98	52.83	59.41	4000	250 MB
sentence-transformers/paraphrase-multilingual-mpnet-base-v2 	65.83	41.68	53.75	2500	970 MB
sentence-transformers/paraphrase-albert-small-v2 	64.46	40.04	52.25	5000	43 MB
sentence-transformers/multi-qa-MiniLM-L6-cos-v1 	64.33	51.83	58.08	14200	80 MB
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 	64.25	39.19	51.72	7500	420 MB
sentence-transformers/paraphrase-MiniLM-L3-v2 	62.29	39.19	50.74	19000	61 MB
sentence-transformers/distiluse-base-multilingual-cased-v1 	61.30	29.87	45.59	4000	480 MB
sentence-transformers/distiluse-base-multilingual-cased-v2 	60.18	27.35	43.77	4000	480 MB

The choice was based on speed/performance trade-off. We've additionally decided to compare **LogisticRegression** classification head with the **LightGBM** based one. Afterwards we've conducted 4 additional training runs with **optuna**. Both 20 trial long.

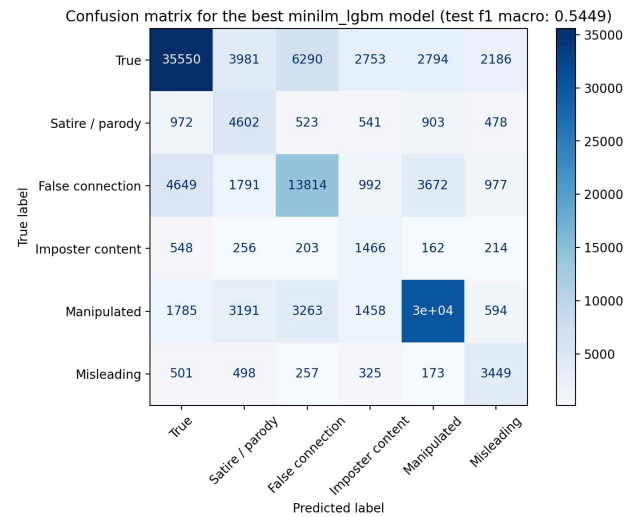
Below are confusion matrices for each of the approaches:



paraphrase-MiniLM + LR

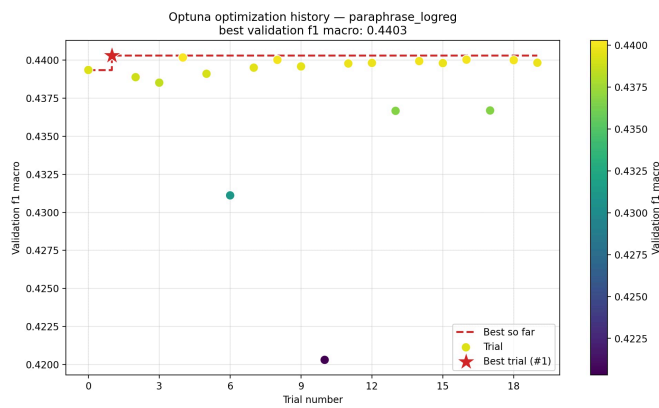


all-MiniLM + LR

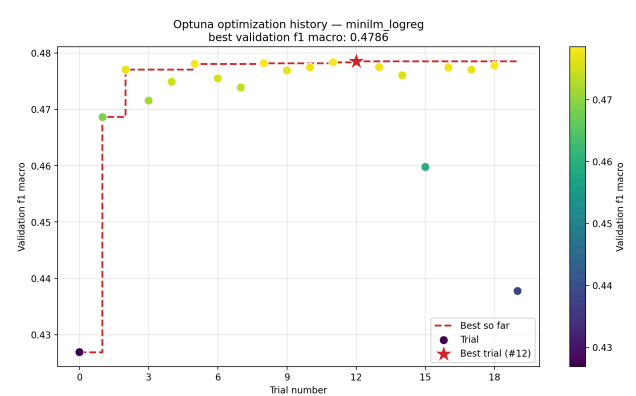


And the optuna training history (val f1 / trial graph)

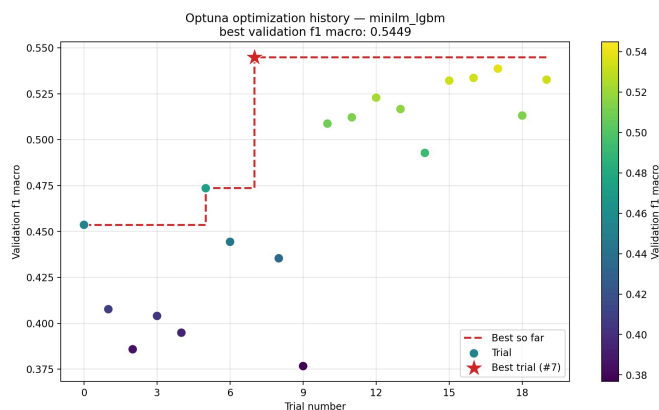
paraphrase-MiniLM + LR



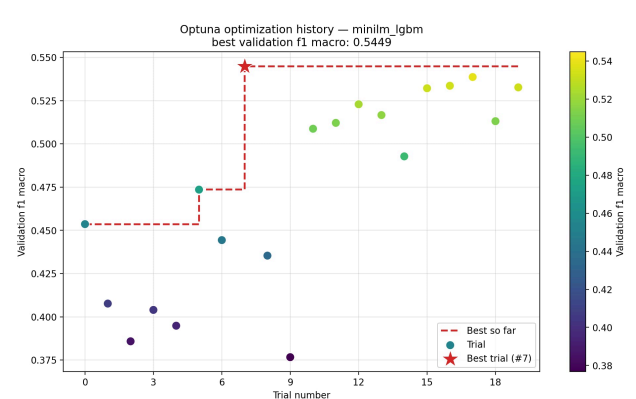
all-MiniLM + LR



paraphrase-MiniLM + LightGBM



all-MiniLM + LightGBM



name	accuracy	macro precision	macro recall	macro f1
baseline	0.658	0.534	0.641	0.558
paraphrase_logreg	0.536	0.444	0.551	0.443
paraphrase_lgbm	0.609	0.585	0.585	0.500

name	accuracy	macro precision	macro recall	macro f1
minilm_logreg	0.582	0.469	0.579	0.479
minilm_lgbm	0.655	0.520	0.616	0.545

The results we got using semantic embeddiings were actually worse than in the case of the baseline (in terms of Macro F1 score). Nonetheless we believe they could generalize better since they do not suffer from the same data leakage problem.

Step III. The VLM (Vision-Language Model)

Gathering images

Along the original dataset over 110GB of image data was shared by the authors. This made it impractical to work with in the cloud computing environments we depended on (kaggle and google colab).

Thus we've decided go along with fetching the images off of the internet by ourselves. We have used `image_url` column located for the each sample.

There were two great hinderences we've encountered:

- some images were no longer available
- rate limiting

nonetheless we've accepted that limitation and went further. To save on storage space we've been continuously writing to a parquet file data in shape

```
{
  id: str,
  image: bytes,
  clean_title: str,
  split: str,
  6_way_label: int,
}
```

where images were resized to 512x512 and stored as binary representation of a jpeg file. This has saved us tons of storage. A file containing over 260k images took around 9.7GB of storage.

Evnetually storing all the images in this format would be a feasible future direction of the project.

VLM-based model

To make use of visual data we have used `Qwen3-VL-Embedding-2B` model to extract embeddings off of both modalities.

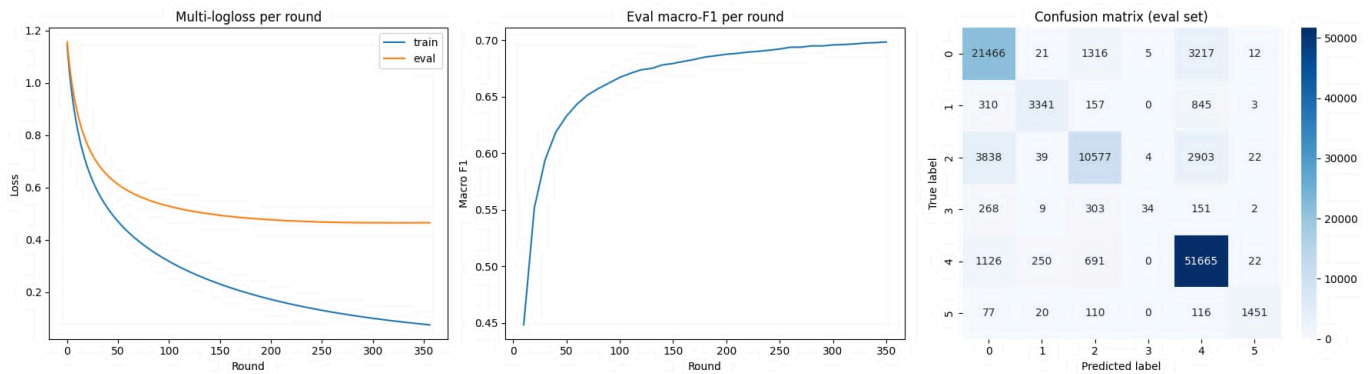
```
def embed_batch(clean_titles: list[str], image_bytes_list: list[bytes]) ->
np.ndarray:
    """Returns float32 array of shape [batch_size, 2048]."""
    inputs = [
```

```

{"text": title or "", "image": Image.open(BytesIO(img)).convert("RGB")}
for title, img in zip(clean_titles, image_bytes_list)
]
return model.encode(inputs, convert_to_numpy=True, batch_size=len(inputs))

```

These embeddings were then fed into LightGBM classification head. We've went with LightGBM as it seemed to score much better than logistic regression-based methods from the previous trials.



On the reduced dataset with `(image, clean_title)` pairs we have achieved:

- accuracy: 0.816
- macro F1: 0.764
- macro precision: 0.815
- macro recall: 0.739

which is much better result than for any of the previous approaches. Visual data in fact did help.

Implications and Future Directions

Next steps that could be taken are:

1. Extend parquet dataset to contain all the images
2. Try different classification heads.
3. Use metadata such as scores, author names.